

Exam 06: mini_serv

Mapa Mental y Pseudocódigo

Advertencia Importante

Recordatorio Importante: La macro `FD_ISSET` no devuelve estrictamente un booleano (`true/false`). Nunca uses `== true`. Usa siempre `if (FD_ISSET(fd, &set))` o `!= 0`.

1. Preparación e Inicialización

- Copiar funciones dadas en el subject: `extract_message` y `str_join`.
- Crear estructuras:
 - `t_clients`: Arrays para almacenar los mensajes parciales (`char *`), los IDs de clientes (`int`) y una variable para el ID actual (consecutivo).
 - `t_server`: Struct general que contiene los clientes, los conjuntos de descriptors para multiplexing (`read_fds`, `wrtefds`, `bkp_fds`) y el `fd_socket` del servidor.
- Crear función `msg_err()`: Ejecuta `write(2, "Fatal error\n", 12)` y termina el programa con `exit(1)`.
- Crear función `broadcast(sender_fd, str, server)`: Itera sobre todos los file descriptors y usa `send()` para enviar el mensaje a todos **excepto** al `sender_fd` y al `fd_socket` del servidor. (Verificando siempre que el FD destino esté encendido en `wrtefds`).

2. Configuración del Servidor (MAIN)

- **Control de Argumentos:** Si `argc != 2`, imprimir "Wrong number of arguments\n" por `STDERR` y `exit(1)`.
- **Declaraciones de Variables:** Extraer puerto (`atoi`), declarar e inicializar a cero struct `t_server` y `sockaddr_in`.
- **Inicializar Red:** Usar `bzero` en `sockaddr_in`, configurar familia `AF_INET`, IP fija a `127.0.0.1` (`htonl(2130706433)`) y aplicar el puerto con `htons()`.
- **Crear Socket:** Inicializar con `socket(AF_INET, SOCK_STREAM, 0)`. Si devuelve error (`-1`) -> `msg_err()`.
- **Binding:** Unir el FD del socket con la configuración de red (IP y Puerto). Si falla -> `msg_err()`.
- **Listen:** Poner el servidor a escuchar conexiones entrantes (`backlog` ej. 10). Si falla -> `msg_err()`.
- **FD Master:** Limpiar con `FD_ZERO` el set maestro (`bkp_fds`) e insertar el socket del servidor usando `FD_SET`.

3. Bucle Principal (El multiplexing)

Iniciamos un while(1) que mantendrá al servidor operando continuamente.

- Resetear siempre read_fds y writefds copiándolos desde bkp_fds en cada iteración.
- Llamar a la función bloqueante select(FD_SETSIZE, &read_fds, &writefds, NULL, NULL). Si falla (-1), usar continue.

3.1. Gestión de Nuevas Conexiones (Server Socket)

- Comprobar si el socket del servidor tiene actividad: if (FD_ISSET(server.fd_socket, &server.read_fds)).
- Aceptar la nueva conexión usando accept().
- Asignar al nuevo cliente su ID consecutivo (guardándolo en el array) y aumentar en 1 el contador global.
- Registrar el FD del nuevo cliente en el set maestro: FD_SET(fd, &bkp_fds).
- Formatear con sprintf el mensaje "server: client X just arrived\n" y distribuirlo a todos llamando a broadcast().

3.2. Gestión de Clientes Conectados

- Crear bucle para iterar los FD: desde i = 3 hasta FD_SETSIZE.
- Saltar la iteración si el índice es igual al socket del servidor: if (i == server.fd_socket) continue;.
- Verificar si el cliente tiene actividad: if (FD_ISSET(i, &server.read_fds)).
 - Llamar a recv(i, buf, sizeof(buf) - 1, 0) para leer los datos entrantes.
 - **Caso A: Desconexión o Error (recv <= 0)**
 - Formatear mensaje "server: client X just left\n" con su ID y enviarlo por broadcast().
 - Comprobar si quedó algún mensaje parcial en memoria. Si es así, free() y apuntar el puntero a NULL.
 - Cerrar la conexión ejecutando close(i).
 - Eliminar el FD del set maestro: FD_CLR(i, &bkp_fds).
 - **Caso B: Mensaje Recibido (recv > 0)**
 - Asegurar que el buffer termine en caracter nulo agregando \0 al final.
 - Reconstruir el mensaje completo uniendo lo guardado anteriormente y el buffer nuevo: str_join(parcial, buf).
 - Verificar fugas: Si str_join devuelve null, lanzar msg_err().
 - **Bucle de extracción de líneas completas** con while(extract_message(...) == 1):
 - Crear el prefijo de emisor con sprintf("client %d: ", id).
 - Enviar el prefijo mediante broadcast().
 - Enviar la línea de texto mediante broadcast().
 - Liberar la memoria de la línea extraída con free(line).